
SEQUENCE TO SEQUENCE LEARNING USING NEURAL NETWORKS

Sounak Das
MB2331
M.Stat Year I
Indian Statistical Institute
sounak670@gmail.com

Souvik Roy
MB2332
M.Stat Year I
Indian Statistical Institute
souvik.roy.pi@gmail.com

Tarun Agarwal
MB2333
M.Stat Year I
Indian Statistical Institute
tarun.agarwal2552@gmail.com

1 Introduction

In the expansive realm of artificial intelligence, this project embarks on the challenge of harnessing the capabilities of recurrent neural networks for sequence-to-sequence learning. The primary objective is to equip neural networks with the proficiency to adeptly process sequences of variable lengths, particularly focusing on tasks demanding intricate transformations, such as language translation.

This endeavor becomes particularly salient in the context of language translation, exemplified here by the conversion of English sentences into their Hindi counterparts. The project leverages the inherent strengths of recurrent neural networks, specifically tailored to discern and capture sequential dependencies, facilitating a nuanced understanding of contextual information within sequences of varying lengths.

The exploration unfolds with a spotlight on the translation domain, where the model endeavors to decode the intricacies of syntax and semantics across language pairs of diverse lengths. Beyond its immediate applications, the project aspires to contribute insights and methodologies that transcend language translation, promising to reshape the landscape of natural language processing and other domains reliant on the nuanced handling of sequential data.

2 Universal Approximation Theorem

Quoting [1], neural networks are just a means to estimate non linear continuous functions in models like $\mathbb{E}[\vec{Y}|X] = f(X)$. Most neural networks apply predefined functions like the sigmoid or the ReLu to linear functions of the inputs and aim to learn the appropriate linear coefficients. In this section, we try to establish why this estimation procedure is an ideal one.

Assume for simplicity that the unknown function we wish to model is a continuous function on $\mathbb{R}^n$. X contains the input vectors $\vec{x}_i$ as columns, and $f(X)$ denotes the matrix with columns $f(\vec{x}_i)$. In this setup, we have the following theorem that appeared in [2].

Universal Approximation Theorem: A continuous function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is not a polynomial if and only if for every $\varepsilon > 0$, $n, m \in \mathbb{R}$, $K \subset \mathbb{R}^n$ and K compact, $f : K \rightarrow \mathbb{R}^m$, f continuous, there exist $k \in \mathbb{N}$, $\mathbf{A} \in \mathbb{R}^{k \times n}$,

$\vec{b} \in \mathbb{R}^k$, $\mathbf{C} \in \mathbb{R}^{m \times k}$ such that

$$\sup_{\vec{x} \in K} \|f(\vec{x}) - g(\vec{x})\| < \varepsilon,$$

where $g(\vec{x}) = \mathbf{C}\sigma(\mathbf{A}\vec{x} + \vec{b})$.

Using the above theorem, we can approximate any continuous function on a compact interval by a continuous non polynomial function with as much accuracy as we please. The only unknowns here are the matrices A and C ¹. Neural networks aim to learn these unknown matrices while using suitable continuous functions for σ . It needs to be observed that in principle we can use any function in place for σ as long as it is not continuous and not a polynomial, but if we add some useful properties to this function, boundedness for example, then we can use it to model probabilities as well. The most commonly used functions for σ are the sigmoid, tanh and the ReLu whose closed forms shall be mentioned later in this report. These functions are often called **activation functions** in computer science vocabulary. We now turn our attention to the estimation of the matrices A and C . Most computer scientists call these matrices the **weight** matrices since they correspond to the coefficients or the "weights" in the linear combinations of the input X .

Since we already have made the approximation of f (using our choice of σ) pretty accurate, we shall have to estimate the the weights pretending that the actual non linear function was indeed σ . It makes sense to assume the usual regularity conditions on the distribution of the data here. More specifically, we assume that the distribution is identifiable, and hence we have the MLE's of the parameters, and any functions thereof, to be consistent. Therefore, if $\hat{C}$ and $\hat{A}$ are the MLE's of the matrices C and A under the distribution of the data and the model that the true unknown function is indeed σ , we shall have,

$$\|\hat{C}\sigma(\hat{A}x) - C\sigma(Ax)\| \rightarrow 0$$

in probability. Therefore, we can use $\hat{C}\sigma(\hat{A}x)$ to estimate f to any degree of accuracy as we please. Most neural networks shall approximate the function f in layers, wherein the first layer shall be our σ and the i th layer shall be σ^i . It is easy to see that more layers increase the degree of the accuracy to which we want to predict f , but since we also run the risk of overfitting, the number of layers is usually kept between 100 to a few thousand.

3 Estimation

While estimating the weights as mentioned in the previous section, most neural networks minimise a loss to obtain the optimal estimates. The most natural loss is the negative of the log likelihood, which is minimised at the MLE which we know is consistent. In this section we shall talk about the approach that is used most widely to minimise the loss and the associated problems with it. We shall assume that the concerned loss is $\sum (y_k - f_k(x))^2$ which is minimised at the MLE if the underlying distribution is Gaussian. This is a commonly occurring case, and we use this to illustrate the gradient descent in a manner similar to [1].

Suppose that all the unknown parameters are put together in θ and the sample loss function is therefore,

$$R(\theta) = \sum_{k=1}^K \sum_{i=1}^N (y_{ik} - f_k(x_i))^2.$$

Consider that θ is made up of two blocks of parameters α and β . The first layer of the neural network is $Z = \sigma(W_\alpha^T \vec{x})$ ², the next layer is $T = W_\beta^T Z$ and the final layer is $f_k(X) = g_k(T)$ with g_k being another non linear continuous

¹we shall henceforth ignore the bias b for simplicity as it can be easily absorbed in the matrix A

²All the parameters for the α can be taken together in the weight matrix W_α

activation function. Most texts such as [3] use definition of a neural network in terms of its *pre-synaptic* and *post-synaptic activation* functions. These are as follows:

$$\vec{a}^{(k)} = \mathbf{W}^{(k)} \vec{y}^{(k-1)}$$

and

$$\vec{y}^{(k)} = \sigma(\vec{a}^{(k)}).$$

³ The loss $R(\theta)$ can be written as follows.

$$R(\theta) = \sum_{i=1}^N R_i = \sum_{i=1}^n \sum_{k=1}^K (y_{ik} - f_k(x_i))^2,$$

with derivatives,

$$\frac{\partial R_i}{\partial \beta_{km}} = -2(y_{ik} - f_k(x_i))g'_k(\beta_k^T z_i)z_{mi}, \quad (1)$$

$$\frac{\partial R_i}{\partial \alpha_{ml}} = -\sum_{k=1}^K 2(y_{ik} - f_k(x_i))g'_k(\beta_k^T z_i)\beta_{km}\sigma'(\alpha_m^T x_i)x_{il}. \quad (2)$$

Now, we use the following iterative rule to obtain the weights α and β .

$$\beta_{km}^{(r+1)} = \beta_{km}^{(r)} - \gamma_r \sum_{i=1}^N \frac{\partial R_i}{\partial \beta_{km}^{(r)}} \quad (3)$$

$$\alpha_{ml}^{(r+1)} = \alpha_{ml}^{(r)} - \gamma_r \sum_{i=1}^N \frac{\partial R_i}{\partial \alpha_{ml}^{(r)}}, \quad (4)$$

where γ_r is a learning rate that is usually small and fixed. Now, we write (1) and (2) as

$$\begin{aligned} \frac{\partial R_i}{\partial \beta_{km}} &= \delta_{ki} z_{mi} \\ \frac{\partial R_i}{\partial \alpha_{ml}} &= s_{mi} x_{il}. \end{aligned}$$

The quantities δ_{ki} and s_{mi} are the errors from the current model at the output and the hidden layer units respectively. From their definitions these errors satisfy,

$$s_{mi} = \sigma'(\alpha_m^T x_i) \sum_{k=1}^K \beta_{km} \delta_{ki}.$$

The last equation is known as the backpropagation equation. Using this, the updates in (3) and (4) can be implemented with a two pass algorithm. In the forward pass, the current weights are fixed and the predicted values $\hat{f}_k(x_i)$ are computed from the description of the layers. In the backward pass, the errors δ_{ki} are computed, and then backpropagated via the last equation to give the errors s_{mi} . Both the sets of errors are then used to compute the gradients for the updates in (3) and (4). This two pass procedure is known as backpropagation.

The advantages of backpropagation are its simple local nature. In the backpropagation algorithm, each hidden unit passes and receives information only to and from units that share a connection. Hence it can be implemented efficiently on a parallel architecture computer.

³Here, the superscript k denotes the k th layer. This is demonstrated in figure 1, where the vectors are taken as row vectors instead of column vectors.

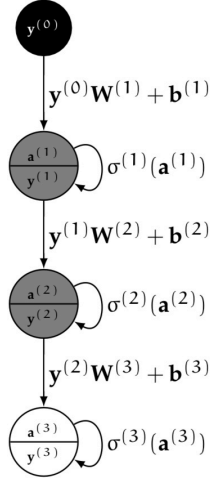


Figure 1: Illustration of a feed forward network when the first layer is omitted. The four layers of nodes are called the input layer(black), hidden layers (gray) and output layer(white). The hidden layer and output layers are divided horizontally into pre-synaptic activations a and post-synaptic activations y . Between each of the layers the affine transformation defined by W and b is applied.

3.1 RNNs

A Recurrent Neural Network (RNN) can be perceived as a temporal extension of a standard neural network. The hidden layers can then forward information in time, which is typically done by letting the pre-synaptic activation at time step t for layer k depend on the layer at the same level at the previous time step, and the previous layer at the same time step. [4] defines the presynaptic activation as an unknown linear combination of the input vector $\mathbf{W}_{in}\vec{x}$ and the postsynaptic activation as the activation function applied onto the presynaptic result. Figure 2 shows the pre and post synaptic operations.

$$\vec{a}_t^{(k)} = \mathbf{W}_{in}^{(k)} \vec{y}_t^{(k-1)} + \vec{y}_{t-1}^{(k)} \mathbf{W}_{rec}^{(k)}$$

and

$$\vec{y}_t^{(k)} = \sigma(\vec{a}_t^{(k)})$$

where, clearly, $[\vec{y}_t]$ is the post synaptic layer and $[\vec{a}_t]$ is the presynaptic layer. Since, in an RNN, there is new information coming in as well as the past outputs are going to be used, there are two sets of weights $\mathbf{W}_{in}$ which is meant for the new information, and $\mathbf{W}_{rec}$ which is meant for the processed information. Details about the structure of RNNs shall be presented in the next section. Since the information from an RNN is fed into itself, the derivatives will also have a dependency on the timesteps across which we compute the gradient. Suppose we use a loss L , then let

$$\delta_t^k = \frac{\partial L}{\partial \vec{a}_t^{(k)}}.$$

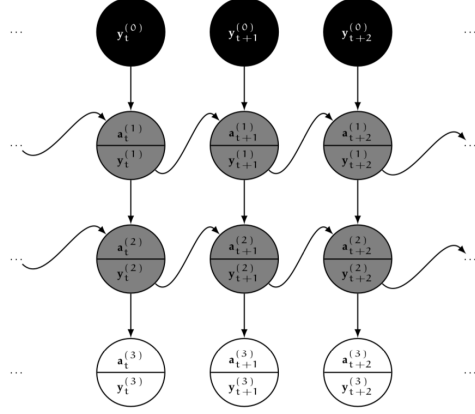


Figure 2: Illustration of an RNN. Compare with figure 1. We have two hidden layers, which have been unfolded in time from left to right. We do not show the operators along the arrows to keep the diagram unclutter.

Then from [4] we have the following,

$$\begin{aligned}
 \delta_t^k &= \delta_{t+1}^k \frac{\partial \vec{a}_{t+1}^{(k)}}{\partial y_t^{(k)}} \frac{\partial y_t^{(k)}}{\partial \vec{a}_t^{(k)}} \\
 &= \delta_{t+1}^{(k)} \frac{\partial \vec{a}_{t+1}^{(k)}}{\partial \vec{a}_t^{(k)}} \\
 &= \delta_K^{(k)} \frac{\partial \vec{a}_K^{(k)}}{\partial \vec{a}_{K-1}^{(k)}} \dots \frac{\partial \vec{a}_{t+1}^{(k)}}{\partial \vec{a}_t^{(k)}}.
 \end{aligned}$$

At this point we already note that the gradients contain the product of $K - t$ jacobians, the exact form of each being the follows :

$$\frac{\partial \vec{a}_{t+1}^{(k)}}{\partial \vec{a}_t^{(k)}} = \text{diag}(\sigma'(\vec{a}_t^{(k)})) \mathbf{W}_{rec}^T$$

. Putting this back at the equation we arrive at

$$\delta_t^{(k)} = \delta_K^{(k)} \prod_{i=t+1}^K \text{diag}(\sigma'(\vec{a}_i^{(k)})) \mathbf{W}_{rec}^T.$$

Using the formulation given in [3], if we consider a linear version of the model (i.e. set σ to the identity function) we can use the power iteration method to formally analyze this product of Jacobian matrices and obtain tight conditions for when the gradients explode or vanish. It is sufficient for $\rho < 1$, where ρ is the spectral radius of the recurrent weight matrix $\mathbf{W}_{rec}$, for long term components to vanish (as $t \rightarrow \infty$) and necessary for $\rho > 1$ for them to explode. Hence, in the training of recurrent neural networks, we often run into the problem of the gradient of the loss function to either vanish or explode. This leads us to the following section on Long Short Term memory networks.

4 Long Short Term Memory Networks

The problem of the vanishing gradient rendered learning of recurrent networks largely impractical on tasks with long term dependencies. Before the discovery by Martens and Sutskever (2011 [5]) that Hessian-free optimisation is capable of dealing with these challenges to some extent as well, LSTM (Hochreiter and Schmidhuber, 1997 [6]) was the method of choice for dealing with time lags spanning hundreds or thousands of steps, for which it was designed. LSTM nevertheless is still the method underpinning most successful applications of recurrent networks on real world

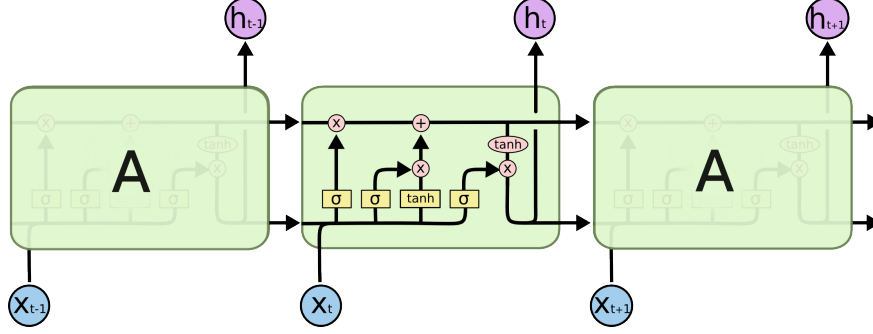


Figure 3: The repeating module in an LSTM contains four interacting layers

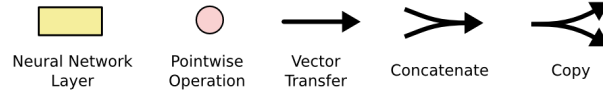


Figure 4: In the above diagram, each line carries an entire vector, from the output of one node to the inputs of others. The pink circles represent pointwise operations, like vector addition, while the yellow boxes are learned neural network layers. Lines merging denote concatenation, while a line forking denote its content being copied and the copies going to different locations.

problems, despite of recent successes to train LSTM-free RNNs on problems with long term dependencies ([7], [8]). It constitutes the state of the art in speech recognition, handwriting recognition and statistical machine translation ([9]). We shall refer to [10] for the inner architecture of an LSTM.

4.1 LSTM architecture

The LSTM has the ability to remove or add information to the cell state, carefully regulated by structures called gates. These gates are nothing but some sigmoid transformations that we shall apply on the input to estimate the non-linear $f(x)$ as in an AR(1) model. Computer scientists view the gates (figure 5) as a way to optionally let information through since the sigmoid gives one a value between 0 and 1, and this might be viewed a proportion of the information that we let through. The weights of the linear combinations that we input are later learned and this can be thought of the machine deciding the important parts of the input. Nevertheless, we know that all this is just a consequence of the universal approximation theorem and thus, we can apply any non polynomial transform as we please (figure 4). An LSTM has three of these gates to protect, and control the cell state, which is the final output from the neuron after applying all the operations.

The first step in our LSTM is to decide what information we're going to throw away from the cell state. this decision is made by a sigmoid layer called the "forget gate layer" 3. It looks at the current cell state $\vec{y}_{t-1}^A$ and $\vec{y}_t^0$ and computes $\vec{y}_t^A = \sigma \left(\mathbf{W}_1 [\vec{y}_{t-1}^A \quad \vec{y}_t^0]^T \right)$

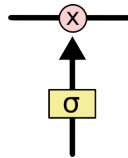


Figure 5: A gate is composed of a sigmoid activation function, and a pointwise multiplication operation

Now, we need to decide what new information we're going to store in the cell state. This has two parts. First, a sigmoid layer called the 'input gate layer' decides which values we'll update. Next, a tanh layer creates a vector of candidate values $\tilde{C}_t$ that could be added to the state. In the next step we'll combine these two to create an update to the state.

$$\begin{aligned}\bar{y}_t^2 &= \sigma \left(\mathbf{W}_2 [\bar{y}_{t-1}^4 \quad \bar{y}_t^{0T}]^T \right) \\ \tilde{C}_t &= \tanh \left(\mathbf{W}_3 [\bar{y}_{t-1}^4 \quad \bar{y}_t^{0T}]^T \right)\end{aligned}$$

It is now time to update the old cell state, $\bar{y}_{t-1}^4$ into the new cell state $\bar{y}_t^4$. This step is a little different from the original RNN architecture that we proposed. We multiply the old state $\bar{y}_{t-1}^3$ by $\bar{y}_t^1$ ⁴ and add $\bar{y}_t^2 \times \tilde{C}_t$, where the multiplication operation is pointwise. Thus, finally the state at time t is

$$\bar{y}_t^3 = \bar{y}_t^1 \times \bar{y}_{t-1}^3 + \bar{y}_t^2 \times \tilde{C}_t.$$

We notice that by the term *state of the cell*, we actually mean $\bar{y}_t^3$. The output that the t th cell produce is $\bar{y}_t^4$ which is also sent to the next cell that we have. Finally we need to decide what we shall output. That can be accomplished by another application of the sigmoid function

$$\vec{o}_t = \sigma \left(\mathbf{W}_4 [\bar{y}_{t-1}^4 \quad \bar{y}_t^{0T}]^T \right)$$

and

$$\bar{y}_t^4 = \vec{o}_t \times \tanh(\bar{y}_t^3).$$

4.2 Loss functions

When we train the neural network, we minimise a loss function. In this section we develop some theory and look at some loss functions to prepare ourselves ahead of the next section that actually deals with how LSTMs solve the vanishing gradient problem.

4.2.1 Maximum likelihood and maximum a posteriori

Given a dataset D_{train} and a probability distribution p parametrised by θ , ML (the maximum likelihood estimator) finds the parameters that make the observed data most likely.

$$ML(\theta) = \arg \max_{\theta} p(D_{train} | \theta).$$

Often we minimise the negative log-likelihood of the data since the data are composed of independent samples. A drawback of MLE is that it is very likely to overfit: given a model of sufficient expressive capabilities, it will converge to the empirical distribution. In fact, maximum likelihood learning equates to minimising the KL-divergence between the model and the empirical distribution [11]. A partial solution is to resort to maximum a posteriori: a prior belief is formulated on the parameters in the form of a density function over the parameter space:

$$\arg \max_{\theta} p(D_{train} | \theta) p(\theta)$$

which after taking the log results in an additional regularisation term:

$$L(\theta) = -\ln p(\theta) - \sum_{i=1}^N \ln p(x_i | \theta)$$

Note that the regularisation term is not part of the sum. Therefore, in the limit of infinite data $N \rightarrow \infty$, maximum a posteriori will converge to the maximum likelihood solution. Nevertheless, it can be shown that the choice of prior and parameterisation are intervened, rendering the method not invariant to reparameterisation. Further, it does not supply a measure of uncertainty. Also, the mode of a distribution is not necessarily a representative summary of a distribution. A common practice therefore is to consider the loss mentioned in the following section.

⁴Even though all of $\bar{y}_t^1$, $\bar{y}_t^2$ and $\bar{y}_t^3$ have the same closed forms, their weights shall decide the operations that we associate with them. Here, the weights $\mathbf{W}_1$ shall be trained so that the gates $\bar{y}_t^1$ are trained to forget the useless information, and the weights $\mathbf{W}_2$ shall be trained so that the gates $\bar{y}_t^2$ are trained to adequately take in new information.

4.2.2 Variational Inference or Minimum Description Length

Consider the sample observed x , with a probability distribution p , and a random variable Z , with a probability distribution q such that $q(z) \approx p(z|x)$. Now, consider the KL divergence between the unconditional distributions p and q ,

$$\mathbb{KL}[q(z)||p(z)] = \int \left(\ln \frac{q(z)}{p(z)} \right) q(z) dz.$$

A simple application of the Bayes Theorem yields,

$$\begin{aligned} \mathbb{KL}[q(z)||p(z)] &= \int \left(\ln \frac{q(z)}{p(z)} \right) q(z) dz \\ &= \int \left(\ln \frac{q(z)p(x|z)}{p(x)p(x|z)} \right) q(z) dz \\ &= \mathbb{KL}[q(z)||p(z|x)] + \int q(z) \ln p(x|z) dz - \ln p(x) \\ \implies -\ln p(x) &= -\mathbb{E}_q[\ln p(x|z)] + \mathbb{KL}[q(z)||p(z)] - \mathbb{KL}[q(z)||p(z|x)] \\ &\leq -\mathbb{E}_q[\ln p(x|z)] + \mathbb{KL}[q(z)||p(z)] = L_{mdl} \end{aligned}$$

L_{mdl} is known as the minimum description length or a variational lower bound, and it being an upper bound to the negative log likelihood, can be considered to be minimised. The variational lower bound is often substantially easier to optimise than the marginal likelihood if the prior $p(z)$ and the variational approximation $q(z)$ are of an adequate form. More specifically, i) the KL divergence between the two and ii) the expected data likelihood (the first term of the RHS) need to be tractable. In the case of the latter, this can be approximated via Monte Carlo if we can efficiently sample from $q(z)$.

4.2.3 Vanishing/Exploding gradient problem resolution

Now that we have several candidates for the loss function, we might choose anyone of them and try to minimise them using the gradient descent. This section presents how the vanishing gradient problem is solved using the architecture of LSTMs. Let $f = \tanh$, and $\phi(x, v) = v \times f(x)$, where as usual in this context, the multiplication is being done pointwise, and unless otherwise specified, σ denotes the sigmoid function. Recalling the definition of y_t^3 , we observe that

$$y_t^3 = \phi(y_{t-1}^3, y_t^1) \times \frac{y_t^1}{f(y_t^1)} + \phi(\tilde{C}_t, y_t^2) \times \frac{\tilde{C}_t}{f(\tilde{C}_t)}.$$

Division in the above is also done pointwise. Hence,

$$\left| \frac{\partial y_t^3}{\partial y_{t-1}^3} \right| = \left| \frac{\partial y_t^3}{\partial y_t^1} \right| \left| \frac{\partial y_t^1}{\partial y_{t-1}^3} \right| \gtrsim |y_t^1| |f'(y_{t-1}^3)|.$$

Thus,

$$\left| \frac{\partial y_t^3}{\partial y_k^3} \right| \gtrsim \prod_{i=1}^{k-t} |y_i^1| |f'(y_i^3)|.$$

This can of course vanish or explode, but it no more depends on the learned weight matrix, and therefore, we can train the weight matrix without any problem.

5 Sequence to Sequence learning

In this section we talk about implementing LSTMs to learn a sequence of variable length from another sequence of variable length. We shall extend the problem to a rather practical one of trying to generate translations from English to Hindi. We shall heavily reference [12] and [13] throughout this section.

5.1 Embedding architecture

Our method[12] uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. We have two jobs at hand, encoding the input sequence to this fixed vector and decoding this correctly to another sequence of variable length. The loss function that we shall consider is negative of the log likelihood on the probability distribution of the words. We shall consider each word as a one hot encoded vector, and shall obtain the vector of each word accordingly. The model is called in [13] as an RNN Encoder Decoder model. We shall use an LSTM instead of an RNN, with a modified architecture.

Let us denote an source phrase by $X = (\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N)$ and a target phrase by $Y = (\vec{y}_1, \vec{y}_2, \dots, \vec{y}_M)$. Each phrase is a sequence of K -dimensional one-hot vectors, such that only one element of the vector is 1 and all the others are 0. The index of the active (1) element indicates the word represented by the vector. Here we use $K = 100$. Each word of the source phrase is embedded in a 100-dimensional vector space: $e(\vec{x}_i) \in \mathbb{R}^{100}$. The hidden state of an encoder consists of 3 hidden units, and each one of them at time t is computed as follows [13]: ⁵

$$\vec{y}_j^t = \vec{z}_j \times \vec{y}_j^{t-1} + (\vec{1} - \vec{z}_j) \times \tilde{h}_j^t,$$

where

$$\begin{aligned} \tilde{h}_j^t &= \tanh \left((\mathbf{W}e(\vec{x}_t))_j + (\mathbf{U}(\vec{r} \times \vec{h}^{t-1}))_j \right) \\ \vec{z}_j &= \sigma((\mathbf{W}_ze(\vec{x}_t))_j + (\mathbf{U}_z\vec{h}^{t-1})_j), \\ \vec{r}_j &= \sigma((\mathbf{W}_re(\vec{x}_t))_j + (\mathbf{U}_r\vec{h}^{t-1})_j). \end{aligned}$$

To make the equations uncluttered, we omit the biases. The initial hidden state $\vec{y}_j^0$ is fixed at 0. Once the hidden state at the step N is computed, the representation of the source phrase c is taken as

$$\vec{c} = \tanh(\mathbf{V}\vec{h}^N).$$

Now, we take a look at the decoder. Since during training the encoder and the decoder learn simultaneously, we shall invoke the modelling of the loss later in the architecture. The decoder starts by initializing the hidden state with

$$\vec{h}'^0 = \tanh(\mathbf{V}'\vec{c}),$$

where we will use $'$ to distinguish parameters of the decoder from those of the encoder. The hidden state at time t of the decoder is computed by

$$\vec{y}_j'^t = \vec{z}_j' \times \vec{y}_j'^{t-1} + (\vec{1} - \vec{z}_j') \times \tilde{h}_j'^t,$$

where,

$$\begin{aligned} \tilde{h}_j'^t &= \tanh \left((\mathbf{W}'e(\vec{x}_t))_j + (\vec{r}' \times (\mathbf{U}'\vec{h}^{t-1} + \mathbf{C}\vec{c}))_j \right) \\ \vec{z}_j' &= \sigma((\mathbf{W}'_ze(\vec{x}_t))_j + (\mathbf{U}'_z\vec{h}^{t-1})_j + (\mathbf{C}_z\vec{c})_j), \\ \vec{r}_j' &= \sigma((\mathbf{W}'_re(\vec{x}_t))_j + (\mathbf{U}'_r\vec{h}^{t-1})_j + (\mathbf{C}_r\vec{c})_j). \end{aligned}$$

We make a couple of observations here. First, we observe that the encoder and decoders used are RNNs that resemble an LSTM each, but are not exactly LSTMs. The main difference between them and an LSTM is the incorporation of the forget gate. The forget gate and the recall gate are here incorporated by the same sigmoid function, and the recall gate is separately incorporated in the the presynaptic layer. In theory, using the vanilla LSTM will also work since we

⁵The subscript denotes the hidden unit/layer which is being computed and the superscript denotes the timestep. Since all the layers are taken as vectors, the subscript can also be thought of as the component in the vector. This is opposite to the notation we have previously used. All multiplications and division operations are being held pointwise.

are after all applying the universal approximation theorem with non trivial activation functions, but the authors of [13] have found considerably better results in language translation using this architecture. We observe that this architecture is also free of the vanishing gradient problem as is the traditional LSTM. [12] present a version of the same work using LSTMs. The second observation that we make is about the word embedding, for which we make a brief digression in the following section.

5.1.1 Word2Vec

The problem of trying to capture the features in the words has been an old one, and numerous work has been done in this area. The most notable ones that we shall refer to here are [14] and [15]. [14] led to the creation of Google's WORD2VEC which is the most commonly used word embedding model nowadays for transfer learning. We here discuss some features of WORD2VEC.

It is essentially the same as our encoder, in the sense that it learns a fixed vector for each word given its context words. Suppose we encode each word previously in a one hot manner. Then, our aim is to just learn a matrix $\mathbf{E}$ such that given the one hot encoding of the word $\vec{o}_w$, the embedding $\vec{e}_w$ can be obtained as

$$\vec{e}_w = \mathbf{E}\vec{o}_w.$$

The most commonly used model to learn this matrix is known as the **skip-gram** model. Define $\mathbf{y}_{1:t}$ as the context of the word $\vec{y}_t$, which is basically nothing but the collection of the first $t - 1$ words in the sequence of words. The first word in the sequence of words is $\langle \text{SOS} \rangle$ which denotes the start of sentence, and the last word is $\langle \text{EOS} \rangle$ denoting the end of sentence. For computational ease, we shall take the length of each sentence to be at most n , and the lengths of the words can be arbitrary. Now, we model the probability of a given word t appearing given the word c is already observed in the context,

$$\mathbb{P}(\vec{y}_t|\vec{y}_c) = \frac{e^{\vec{\theta}_t^T \vec{e}_c}}{\sum_{i=1}^{|V|} e^{\vec{\theta}_i^T \vec{e}_c}}.$$

Here V denotes the set of all the words, known as the vocabulary, and $\vec{\theta}_t$ is the parameter for the word t . We shall now maximise the probability of the word $\vec{y}_t$ appearing given the context words $\vec{y}_c$ and consequently learn the matrix $\mathbf{E}$. In our task, we shall not learn the matrix $\mathbf{E}$ separately and use that to decode. Instead, we shall learn the decodings first and the matrix shall appear as a by product.

5.2 Loss

We shall use the negative of the log likelihood

$$-\ln \mathbb{P}(\mathbf{Y}_n|\mathbf{X}_n).$$

The above can be written as

$$-\sum_{t=1}^n \ln \mathbb{P}(\vec{Y}_t|\mathbf{Y}_{1:t}, \mathbf{X}_n).$$

Each of the above terms in the sum can be sought to be minimised separately, which is indeed what we shall have to do if we aim to get a word embedding. But since here we have a supervised framework, we shall model each of the above probabilities also conditioned on the source sequence $\mathbf{X}_n$ as follows:

$$\mathbb{P}(\vec{Y}_t|\mathbf{Y}_{1:t}, \mathbf{X}_n) = \frac{e^{\vec{g}_t^T \vec{s}_t}}{\sum_{i=1}^{|V|} e^{\vec{g}_i^T \vec{s}_t}} \quad (5)$$

where, contrary to the WORD2VEC model, we have assumed the **entire** context to be given rather than a single word. Here, $\vec{s}^t$ contains the information of all the previous words, and it is defined as [13]

$$\vec{s}^t = \mathbf{O}_h \vec{h}^t + \mathbf{O}_y \mathbf{y}^{t-1} + \mathbf{O}_c \vec{c}.$$

welcome स्वागतम् ।
 how are you? तुम कैसे हो ?
 can you help me here? क्या तुम मेरी मदद कर सकते हो ?
 will he come tomorrow वह कल आएगा क्या ?
 you should have listened to me तुम्हें मेरी बात सुन लेनी चाहिए थी ?
 how many times do I have to tell you that? मुझे अपने बात वाले हैं ?
 why were you late this morning? तुम आज सुबह देर से क्यों आए ?

Figure 6: *The translations for some common phrases*

Now, to estimate the parameters, we simply minimise the loss function after plugging in the model, and applying gradient descent. The following section presents the experimental analysis.

6 Experimental analysis

We have used a dataset of 15K English sentences and their corresponding Hindi translations. The dataset has been obtained from the Tatoeba project. The bilingual corpora contains close to 4K distinct english words, and 3.5K distinct Hindi words. To train the language model, we have capped the vocabulary to 15K words each. The following sections outline the exact procedure.

6.1 Data Preprocessing

We shall first normalize all the english sentences, by adding an $\langle SOS \rangle$ to the beginning and an $\langle EOS \rangle$ at the end. We also convert all the uppercase alphabets by their corresponding lower case. Also we want our model to work to provide translations for words it hasn't seen in the vocabulary, and hence we also include a word $\langle UNK \rangle$ in the english words, which would denote any unknown word that the model has not seen. Now, we encode all the words from both the language with their one hot encoded vectors and initialise the parameters with random values close to zero. We also look at the word embedding vectors $e(t)$ as defined in the previous section, and calculate them as per $e(t) = Eo_t$, where o_t is the one hot encoding of the word, and E has been initialised by random values close to zero.

6.2 Training and results

The training is done on an eight core CPU and took approximately an hour on an average. Once the training is done, we generate the new outputs from the source text, by first feeding in the source vector, and then for the t th position, we look at the predicted probability of it occurring in that position given $y_{1:t}$. The word that is printed is the one with the highest probability given the previous words. We use (5) to obtain the probability of each word given the previous words from the learnt vectors of parameters. We keep on generating words as the word $\langle EOS \rangle$ does not have the highest probability of occurring. We cap the sentence at 100 characters so that, in the case that within the first 100 characters $\langle EOS \rangle$ does not occur, we terminate the sentence and move to the next one. If there are multiple words with the same highest probability we display anyone of them. Some of the translations obtained are shown in 6. Since the dataset is very small we have only tested on very basic translations.

7 Conclusion

In this project, we explored a new neural network architecture, called an RNN EncoderDecoder that is able to learn the mapping from a sequence of an arbitrary length to another sequence, possibly from a different set, of an arbitrary length. The RNN Encoder Decoder is able to either score a pair of sequences (in terms of a conditional probability) or generate a target sequence given a source sequence. Along with the new architecture, we also explored a novel hidden unit that includes a reset gate and an update gate that adaptively control how much each hidden unit remembers or forgets while reading/generating a sequence. Finally, we used this novel encoder decoder framework to translate source sentences in English to Hindi. The cleaned dataset and the code can be found [here](#), and the original collection of datasets can be found [here](#).

References

- [1] Rob Tibshirani, Jerome Friedman, and Trevor Hastie. *Elements of Statistical Learning*. Springer Texts, 2001.
- [2] Ken-Ich Funahashi. On the approximate realization of continuous mappings by neural networks. *Neural Networks*, 2 (3): 183192. doi:10.1016/0893-6080(89)90003-8. ISSN 0893-6080, 1989.
- [3] Razvan Pascanu, Tomas Mikolov, and Y. Bengio. On the difficulty of training recurrent neural networks. *30th International Conference on Machine Learning, ICML 2013*, 11 2012.
- [4] Justin Bayer. Learning sequence representations. *Diss. München, Technische Universität München, Diss.*, 2015.
- [5] Martens J and Sutskever I. Training deep and recurrent networks with hessian free optimization. *Neural Networks: Tricks of the Trade, Springer Berlin*, 479-535. https://doi.org/10.1007/978-3-642-35289-8_27, 2012.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9:1735–80, 12 1997.
- [7] I. Sutskever, J. Martens, G. Dahl, and G. Hinton. On the importance of initialization and momentum in deep learning. *30th International Conference on Machine Learning, ICML 2013*, pages 1139–1147, 01 2013.
- [8] Ilya Sutskever, James Martens, and Geoffrey Hinton. Generating text with recurrent neural networks. pages 1017–1024, 01 2011.
- [9] Alex Graves. Generating sequences with recurrent neural networks. 08 2013.
- [10] Christopher Olah. Understanding lstm networks. colah.github.io, 2015.
- [11] Barber David. Bayesian reasoning and machine learning. *Cambridge University Press*, 2012.
- [12] Sutskever Ilya, Vinyas Oriol, and Quoc V.Le. Sequence to sequence learning with neural networks. *arXiv*, 2014.
- [13] Cho et al. Kyunghyun. Learning phrase representations using rnn encoderdecoder for statistical machine translation. *arXiv*, 2014.
- [14] Mikolov et al. Tomas. Efficient estimation of word representations in vector space. *arXiv: 13801.3781*, 2013.
- [15] Y. Bengio, Réjean Ducharme, and Pascal Vincent. A neural probabilistic language model. volume 3, pages 932–938, 01 2000.